

Validación y Verificación

Técnicas de pruebas dinámicas



Cátedra Ingeniería de Software

Ingeniería en Sistemas de Información - UTN – F. R. Rosario

Referencias

- **El arte de probar el software** – Myers - 1979
- The Art of Software Testing – Myers - 2004
- **Guías INTECO de VAL y VER**
 - `guia_de_validacion_y_verificacion`
 - `guia_de_mejores_practicas_de_calidad_de_producto`
- Otros activos de INTECO (Servicio repositorio documental)
- Guías de RUP
- Guía para la integración de procesos y la mejora de productos de CMMI - `cmmi-dev-v1.2`

Objetivos de esta presentación:

Técnicas de prueba dinámicas

- Basadas en la especificación
- Basadas en la estructura
- Basadas en la experiencia

Pruebas por dimensión de calidad

Técnicas de pruebas dinámicas

Basadas en la especificación (caja negra)

- Particionamiento de equivalencia
- Análisis de frontera
- Tablas de decisión
- Transición de estados
- Pruebas de Casos de uso

Basadas en la estructura (caja blanca)

- Pruebas de sentencia
- Pruebas de decisión
- Prueba de caminos

Basadas en la experiencia

- Adivinar errores
- Pruebas exploratorias

Técnicas basadas en la especificación

- **Particionamiento de equivalencia**
- Análisis de frontera
- Tablas de decisión
- Transición de estados
- Pruebas de Casos de uso

Particionamiento de equivalencia

- Consiste en dividir un conjunto de condiciones de prueba en grupos que puedan ser considerados iguales por el sistema
- Requiere probar sólo una condición para cada grupo o partición.
- Todas las condiciones de una partición serán tratadas de la misma manera por el software.
 - Si una condición en una partición funciona, asumimos que todas las condiciones en esa partición funcionarán
 - Si una de las condiciones en una partición no funciona, asumimos que ninguna de las condiciones en esa partición funcionará.

Particionamiento de equivalencia

- Las particiones de equivalencia se conocen también como clases de equivalencia
- Existen varios tipos de particiones entre las que destacamos:
 - Las particiones de entradas
 - Agrupación en particiones de las posibles entradas del programa
 - Las particiones de salidas
 - Agrupación en particiones de las posibles salidas del programa

Ejemplo de Particiones de entradas

- Entrada válida:
 - enteros en el rango 100-999.
- Particiones válidas
 - 100-999 incluidos.
- Particiones no válidas
 - menor de 100
 - mayor de 999
 - números reales (decimales)
 - caracteres no numéricos.

Ejemplo de Particiones de salidas

- Supongamos un programa que genera las siguientes salidas:
 - 0.5 % de interés por los primeros 1.000 \$
 - 1% de interés por los siguientes 1.000 \$
 - 1.5% para el resto.
- Se pueden definir casos que generen cada una de estas salidas
 - Un valor de entrada entre 0-1000 \$ generaría el 0.5%
 - Un valor entre 1001-2000 \$ generaría el 1%
 - Un valor mayor que 2000 \$ generaría 1.5%

Directrices de Particionamiento de equivalencia

- Si una condición de entrada especifica un **rango de valores** (por ejemplo, el programa "acepta valores de 10 a 100") se identifican:
 - una partición de equivalencia válida (de 10 a 100)
 - dos particiones de equivalencia no válidas (menor que 10 y mayor que 100).

Directrices de Particionamiento de equivalencia

- Si una condición de entrada especifica un **conjunto de valores** (por ejemplo, “una tela puede ser de varios colores: ROJO, BLANCO, NEGRO, VERDE, MARRÓN”), se identifican:
 - una partición de equivalencia válida (los valores válidos)
 - una partición de equivalencia no válida (todos los demás valores no válidos).
- Cada valor de la partición de equivalencia válida se debe manejar de modo diferente.

Directrices de Particionamiento de equivalencia

- Si la condición de entrada se especifica como una **situación de obligación** (por ejemplo, "la cadena de caracteres de entrada debe ser en mayúsculas") se identifican:
 - una partición de equivalencia válida (caracteres en mayúsculas)
 - una partición de equivalencia no válida (todas las demás entradas excepto los caracteres en mayúsculas).

Técnicas basadas en la especificación

- Particionamiento de equivalencia
- **Análisis de frontera**
- Tablas de decisión
- Transición de estados
- Pruebas de Casos de uso

Análisis de frontera

- A esta técnica también se la llama Análisis de valores límite
- El **análisis del valor frontera** está basado en probar los **valores frontera de las particiones**
- En esta técnica también contamos con fronteras válidas (en las particiones válidas) y fronteras no válidas (en las particiones no válidas).

Análisis de frontera

- Los errores tienden a agruparse alrededor de las fronteras
- Por ejemplo, si un programa debería aceptar una secuencia de números entre 1 y 10, el error más probable será:
 - en los valores justo fuera del rango (0 y 11) sean aceptados
 - o en los valores justo en los límites del rango (1 y 10) sean rechazados.

Ejemplo de Análisis de frontera

Consideremos una impresora que tiene una opción de entrada del número de copias a hacer, de 1 a 99.

No válido	Válido	Válido	No válido
0	1	99	100

- Tenemos la partición de enteros de 1 al 99, que tiene como valor mínimo el 1 y como máximo el 99.
- Al 1 y al 99 se les denomina valores frontera válidos porque son fronteras dentro de la partición válida.
- La frontera de los valores no válidos menores es el cero porque es el primer valor que se encuentra cuando se sale fuera de la partición por la parte más baja.
- Por la parte superior del rango también tenemos un valor frontera no válido, el 100.

Directrices de Análisis de frontera

- Para un entero, si el rango de entrada válido es de 10 a 100, probar 9, 10, 100, 101.
- Si un programa espera una letra en mayúsculas, probar de la A a la Z.
- Si la entrada o la salida de un programa es un conjunto ordenado, conviene prestar atención al primer y al último elemento del conjunto.

Directrices de Análisis de frontera

- Si el programa acepta una lista, probar los valores de la lista. Todos los demás valores no son válidos.
- Se ingresa un importe de una moneda, y la denominación de moneda más pequeña es un céntimo. Si el programa acepta un rango específico de la “a” a la “b”, se debe probar “a - 0.01” y “b + 0.01”.
- Para una variable con varios rangos, cada rango es una clase de equivalencia. Si los rangos no están solapados, probar los valores de los límites, después del límite superior y antes del límite inferior.

Valores especiales de Análisis de frontera

- Estos son algunos ejemplos:
 - Al probar minutos y segundos, se debe probar siempre 59 y 0 como el límite superior e inferior para cada campo, sin tener en cuenta la restricción que tenga la variable de entrada.
 - Al probar la fecha (año, mes y día), se deben incluir numerosos casos de prueba como, por ejemplo, un número de días en un mes específico, el número de días de febrero en año bisiesto o el número de días en años no bisiestos.

Técnicas basadas en la especificación

- Particionamiento de equivalencia
- Análisis de frontera
- **Tablas de decisión**
- Transición de estados
- Pruebas de Casos de uso

Tablas de decisión

- Las técnicas de particionamiento de equivalencia y análisis del valor frontera son aplicadas con frecuencia a situaciones o entradas específicas.
- Sin embargo, si diferentes combinaciones de entradas dan como resultados diferentes acciones, resulta más difícil usar las técnicas anteriores.
- Las técnicas Tabla de decisión y Transición de estados están más orientadas a la lógica o reglas del negocio.

Validación y Verificación - Técnicas de prueba dinámicas

EJEMPLO

Un supermercado tiene un plan de fidelidad que ofrece a todos los clientes. Aquellos que tengan una tarjeta de fidelidad disfrutarán los beneficios de descuentos adicionales en todas las compras (regla 3) o la adquisición de puntos de fidelidad (regla 4), que pueden convertirse en vales para el supermercado o en puntos de planes similares en empresas asociadas. Los clientes sin tarjeta recibirán descuentos adicionales si gastan más de 100€ en una visita al establecimiento (regla 2), de otra forma sólo aplicarán las ofertas especiales para todos los clientes (regla 1).

	Regla 1	Regla 2	Regla 3	Regla 4
<u>Condiciones:</u>				
Clientes sin tarjeta	V	V	F	F
Clientes con tarjeta	F	F	V	V
Descuento extra seleccionado	-	-	V	F
Gasto > 100	F	V	-	-
<u>Acciones:</u>				
No descuento	V	F	F	F
Descuento extra	F	V	V	F
Puntos fidelidad	F	F	F	V

Tablas de decisión

- Una tabla de decisión lista todas las **condiciones de entrada** que pueden ocurrir y todas las **acciones** que pueden surgir de ellas.
- Las **reglas de negocio** se incluyen en la parte de arriba de un extremo a otro.
- Cada columna representa una regla de negocio individual y muestra cómo se combinan las condiciones de entrada para producir acciones.
- De esta manera, **cada columna representa un posible caso de prueba**, ya que identifica ambas entradas y salidas.

Tablas de decisión

- En realidad, **el número de condiciones y acciones puede ser muy largo**, pero normalmente el número de combinaciones que producen una acción específica es relativamente pequeño.
- Por esta razón **no se ponen todas las combinaciones de condiciones posibles** en las tablas de decisión, sino que **se restringe a las que corresponden a las reglas de negocio**.

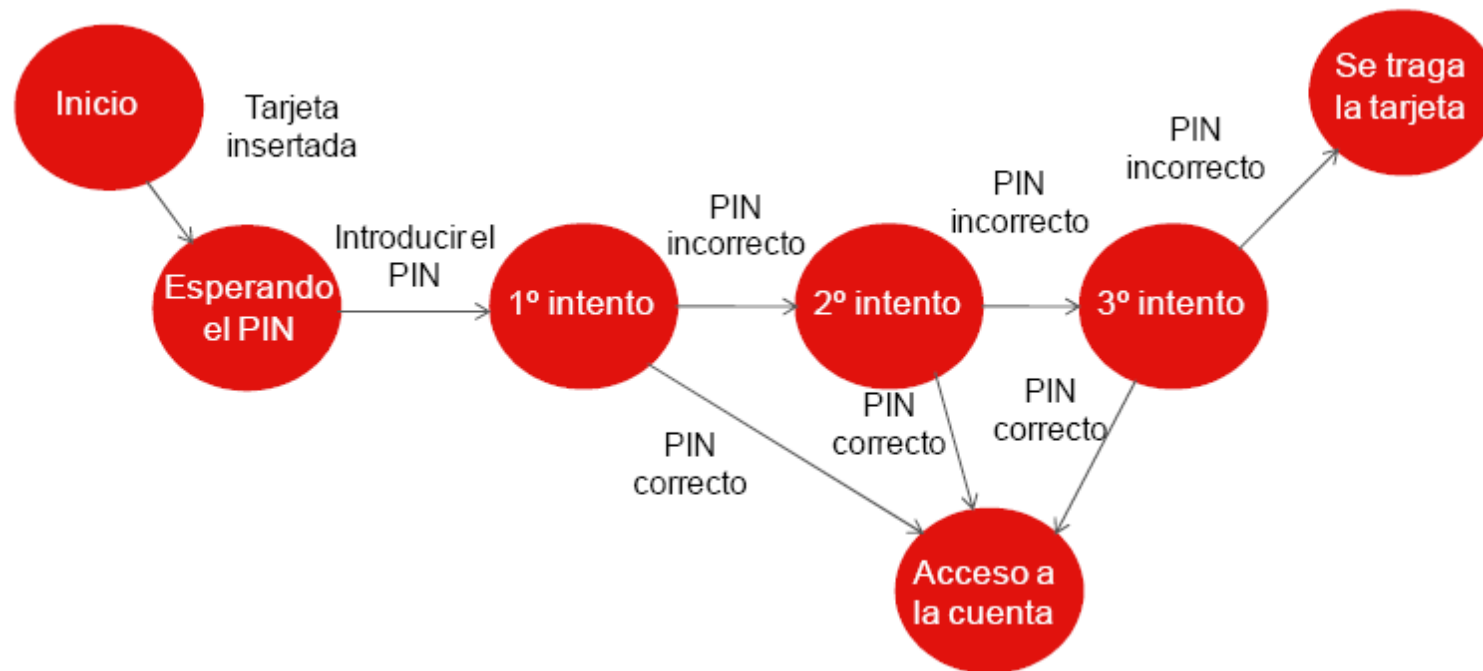
Técnicas basadas en la especificación

- Particionamiento de equivalencia
- Análisis de frontera
- Tablas de decisión
- **Transición de estados**
- Pruebas de Casos de uso

Transición de estados

- Las pruebas de transición de estados se usan cuando alguno de los aspectos del sistema se puede describir en lo que se denomina una “máquina de estados finitos”.
- Esto significa que el sistema puede estar en un número (finito) de estados.
- Este es el modelo en el que se basan el sistema y las pruebas.
- Cualquier sistema en el que se puedan conseguir diferentes salidas con la misma entrada, dependiendo en lo que haya pasado antes, es un sistema de estados finitos.
- Un sistema de estados finitos se suele representar mediante un diagrama de estados.

Ejemplo: Introducir en un Cajero Automático una tarjeta y luego un Número de Identificación Personal (PIN) para acceder a la cuenta de un banco



Análisis del ejemplo de Transición de estados

- No se han especificado todas las posibles transiciones.
 - Habría también un tiempo de espera máximo desde “Esperando el PIN” y desde los tres intentos, desde el que se volvería al estado inicial una vez que el tiempo hubiese pasado y expulsaría la tarjeta
 - Habría una transición desde el estado “Se traga la tarjeta” que volvería al estado inicial.
- No se han especificado todos los eventos posibles
 - Habría una opción de “cancelar” desde “Esperando el PIN” y los tres intentos, desde el que se volvería también al estado inicial y se expulsaría la tarjeta.
- El estado de “Acceso a la cuenta” sería el inicio de otro diagrama de estados que mostraría las transiciones válidas que se podrían llevar a cabo en la cuenta.

Transición de estados

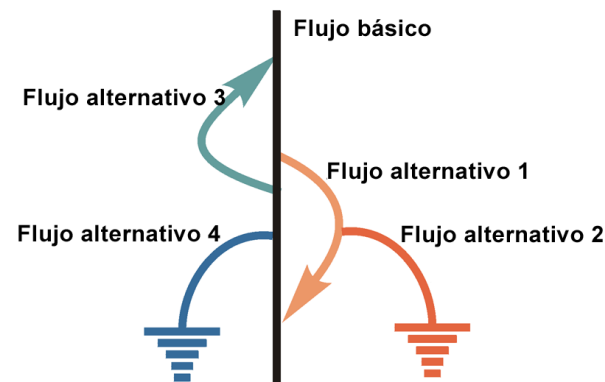
- La obtención de pruebas sólo desde el gráfico de estados es muy bueno para ver las **transiciones válidas**, pero no se ven fácilmente las pruebas negativas, donde se intentan generar **transiciones no válidas**.

Técnicas basadas en la especificación

- Particionamiento de equivalencia
- Análisis de frontera
- Tablas de decisión
- Transición de estados
- **Pruebas de Casos de uso**

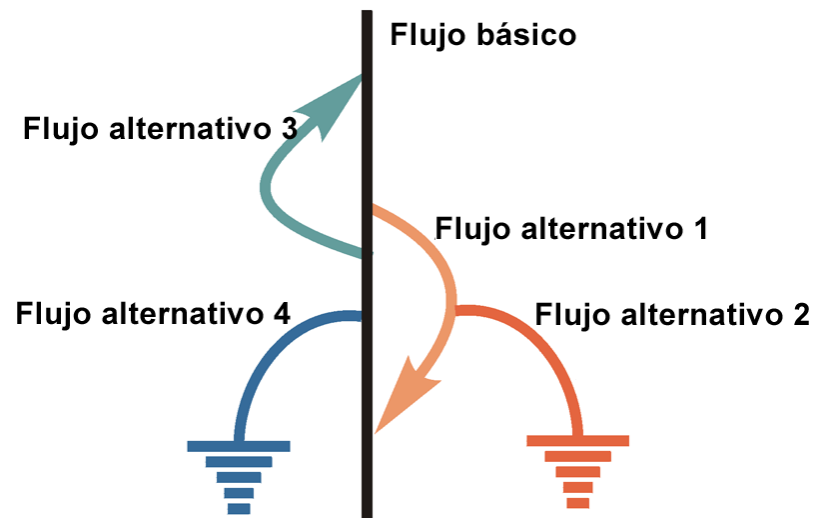
Pruebas de Casos de uso

- Los casos de uso son una forma de especificar la funcionalidad como escenarios de negocio o flujos de procesos
- Las pruebas de caso de uso son una técnica que ayuda a identificar casos de pruebas que ejerciten el sistema entero transición a transición desde el principio al final.



Derivar casos de prueba de casos de uso

- Ver presentación “Derivar casos de prueba de casos de uso”



Técnicas basadas en la estructura

Caja Blanca

- En un sistema donde la seguridad es crítica, es vital estar seguro de que al ejecutar el código no habrá problemas
- Estas técnicas exploran la estructura de los programas.
- En vez de probar el código entero, simplemente se aseguran que una serie de elementos particulares del código funcionan correctamente.
- El punto de partida será el código

Técnicas basadas en la estructura

Caja Blanca

- Basadas en la existencia de tres formas de estructurar código ejecutable:
 - **Secuencia:** Se ejecutan las instrucciones una a una tal y como van apareciendo en el código.
 - **Selección:** Se tiene que decidir si una condición es verdadera o falsa. Si es verdadera, se toma esa ruta, sino se tomaría una ruta diferente.
 - **Iteración:** Consiste en ejecutar un trozo de código más de una vez.

Técnicas basadas en la estructura

- **Pruebas de sentencia**
- Pruebas de decisión
- Prueba de caminos

Pruebas de sentencia

- El objetivo de las pruebas de sentencia es ir probando las distintas sentencias a lo largo del código.
- Si se prueban todas y cada una de las sentencias ejecutables del código, habrá una cobertura de sentencia total.
- Es importante recordar que estas pruebas sólo se centran en sentencias ejecutables a la hora de medir la cobertura.

$$\text{Cobertura} = \frac{\text{Nº sentencias ejecutables ejecutadas}}{\text{Nº sentencias ejecutables}} \times 100$$

Técnicas basadas en la estructura

- Pruebas de sentencia
- **Pruebas de decisión**
- Prueba de caminos

Pruebas de decisión

- El objetivo de estas pruebas es asegurar que las **decisiones** en un programa son realizadas adecuadamente.
- Las decisiones son parte de las **estructuras de selección e iteración**, por ejemplo, aparecen en construcciones tales como: IF THEN ELSE, DO WHILE, etc.
- Para **probar una decisión**, es necesario ejecutarla cuando la condición que tiene asociada es **verdadera** y también cuando es **falsa**. De esta forma, se garantiza que todas las posibles salidas de la decisión se han probado.

Pruebas de decisión

- Al igual que las pruebas de sentencias, las pruebas de decisión tienen una medida de cobertura asociada e intentan conseguir el 100% de la cobertura de decisión

$$\text{Cobertura} = \frac{\text{Nº decisiones probadas}}{\text{Nº decisiones totales en un programa}} \times 100$$

Técnicas basadas en la estructura

- Pruebas de sentencia
- Pruebas de decisión
- **Prueba de caminos**

Prueba de caminos

- Las pruebas de caminos son una estrategia de pruebas estructurales cuyo objetivo es probar cada camino de la ejecución independientemente.
- En todas las sentencias condicionales se comprueban los casos verdadero y falso.
- Las pruebas de caminos no prueban todas las posibles combinaciones de todos los caminos en el programa.

Prueba de caminos

- Para cualquier componente distinto de un componente trivial sin bucles, probar todas las posibles combinaciones de todos los caminos, es un objetivo imposible.
- Existe un número infinito de posibles combinaciones de caminos en los programas con bucles.
- Incluso cuando todas las sentencias del programa se han ejecutado al menos una vez, los defectos del programa todavía pueden aparecer cuando se combinan determinados caminos.

Técnicas de pruebas dinámicas

Basadas en la experiencia

- Adivinar errores
- Pruebas exploratorias

Técnicas basadas en la experiencia

- Las técnicas basadas en experiencias son aquellas a las que se recurre cuando:
 - no hay una especificación adecuada desde la que crear casos de pruebas basados en especificación
 - ni hay tiempo suficiente para ejecutar la estructura completa del paquete de pruebas.
- Las técnicas basadas en experiencia usan la experiencia de los usuarios y de los técnicos de pruebas para determinar las áreas más importantes de un sistema y ejercitar dichas áreas de forma que sean consistentes con el uso que se espera que tengan.

Técnicas basadas en la experiencia

- **Adivinar errores**
- Pruebas exploratorias

Adivinar errores

- Es una técnica que debería usarse siempre como **complemento** de otra técnica más formal.
- El **éxito** de esta técnica depende en gran medida de las **habilidades** de los técnicos de pruebas.
- Hay gente, que por **naturaleza**, es buen técnico de pruebas, y otros que lo son gracias a su **experiencia**.

Adivinar errores

- Una **persona que ha trabajado con un determinado sistema**, conocerá mucho mejor las debilidades que tiene y su forma de trabajar. Por lo tanto, al ejecutar las pruebas oportunas, este técnico podrá adivinar con facilidad cuales son los caminos sobre los que el sistema no trabaja de forma adecuada.
- Hay que **aprovechar las habilidades, la intuición y la experiencia de los técnicos de pruebas** para identificar pruebas especiales que no son fáciles de capturar mediante técnicas más formales

Técnicas basadas en la experiencia

- Adivinar errores
- **Pruebas exploratorias**

Pruebas exploratorias

- Las pruebas exploratorias son una técnica que combina la experiencia de los técnicos de pruebas con una aproximación estructurada a las pruebas
 - cuando faltan de especificaciones o son inadecuadas
 - cuando hay una gran presión respecto al tiempo
- Como su propio nombre indica, las pruebas exploratorias tratan de explorar el software (qué hace, qué no hace, sus debilidades,...)

Pruebas por dimensión de calidad

Dimensiones de calidad

- Funcionalidad (Functionality)
- Usabilidad (Usability)
- Fiabilidad (Reliability)
- Rendimiento (Performance)
- Capacidad de soporte (Supportability)

Para cada una de estas dimensiones, deben implementarse y ejecutarse uno o varios tipos individuales de pruebas

Tipos de prueba por Dimensiones de calidad

- Funcionalidad
 - Prueba de funcionamiento
 - Prueba de seguridad
 - Pruebas de volumen
- Usabilidad
 - Pruebas de usabilidad
- Fiabilidad
 - Prueba de integridad
- Rendimiento
 - Prueba de carga
 - Prueba de estrés
- Capacidad de soporte
 - Prueba de configuración
 - Prueba de instalación

Dimensión Funcionalidad

- **Prueba de funcionamiento**
 - Deben centrarse en verificar los requisitos proporcionados por los casos de uso y las reglas de negocio.
 - Este tipo de pruebas se basa en las técnicas de caja negra; es decir, verifica la aplicación y sus procesos internos interactuando con la aplicación a través de la interfaz gráfica de usuario y analizando la salida o los resultados.

Dimensión Funcionalidad

- **Prueba de seguridad**
 - Garantizar el acceso a los datos o a las funciones del negocio a los actores que corresponda
 - cualquiera puede tener permiso para entrar datos y crear nuevas cuentas, pero sólo los gerentes pueden suprimirlas.
 - el "tipo de usuario A" puede ver toda la información del cliente, incluidos los datos financieros, sin embargo, el "tipo de usuario B" sólo ve los datos demográficos del mismo cliente.
 - Garantizar que sólo los usuarios que disponen de acceso al sistema pueden acceder a las aplicaciones y sólo a través de las caminos apropiados

Dimensión Funcionalidad

- **Pruebas de volumen**
 - Se centran en la verificación de la capacidad del sistema para manejar grandes cantidades de datos, ya sean de entrada y salida o residentes, en la base de datos.
 - Incluye pruebas como la creación de consultas que devolverán el contenido completo de la base de datos, o en las que la entrada de datos tiene la cantidad máxima de datos para cada campo

Dimensión Usabilidad

- **Pruebas de usabilidad**
 - Se encarga de verificar que los usuarios puedan interactuar de la forma mas fácil, cómoda e intuitiva posible.
 - Listas de comprobación INTECO (check list)
 - Pruebas de interfaz de usuario
 - Pruebas de usabilidad de una página WEB

Dimensión Fiabilidad

- **Prueba de integridad**
 - Las pruebas de integridad evalúan la resistencia a los errores
 - frecuencia y gravedad de los errores
 - capacidad de recuperación

Dimensión Rendimiento

- **Prueba de carga**
 - Se encarga de determinar el comportamiento del sistema bajo diferentes cargas de trabajo.
 - Las variaciones de la carga de trabajo suelen incluir la emulación del pico y el promedio de cargas de trabajo que se producen dentro de la operación normal.
 - Se debe verificar el tiempo de respuesta máximo permitido para el número de usuarios contemplado.
 - Listas de comprobación INTECO (check list)
 - Pruebas de rendimiento

Dimensión Rendimiento

- **Prueba de estrés**
 - Determinan el punto de ruptura donde el sistema revela el nivel de servicio máximo que puede conseguir.
 - Pruebas bajo las condiciones de estrés, para observar y registrar el comportamiento que identifique las condiciones bajo las cuales el sistema no puede continuar funcionando adecuadamente:
 - poca memoria o sin memoria disponible en el servidor
 - máximo número de clientes conectados o simulados
 - múltiples usuarios efectúan las mismas transacciones contra los mismos datos o cuentas

Dimensión Capacidad de Soporte

- **Prueba de configuración**
 - Las pruebas de configuración verifican el funcionamiento del sistema en diferentes configuraciones de software y hardware.
 - En la mayoría de entornos de producción:
 - Las especificaciones de hardware de estaciones de trabajo, conexiones de red y servidores de bases de datos varían.
 - Las estaciones de trabajo pueden tener diferentes software cargados (por ejemplo: aplicaciones, drivers, etc.) y, en cualquier momento, muchas combinaciones diferentes pueden estar activas utilizando diferentes recursos.

Dimensión Capacidad de Soporte

- **Prueba de instalación**

- Garantizar que el software se puede instalar bajo diferentes condiciones (como una nueva instalación, una actualización, y una instalación completa o personalizada) bajo condiciones normales y anormales. Las condiciones anormales incluyen espacio en disco insuficiente, falta de privilegios para crear directorios, etc.
- Verificar que, una vez instalado, el software funciona correctamente. Esto habitualmente significa ejecutar una serie de pruebas que se desarrollaron para las pruebas de función.

Dimensiones de calidad / Tipos de prueba

- Check List - INTECO
- Check List - RUP
- Concepto: Tipos de prueba
- Directriz: Técnicas de prueba por riesgo de calidad / tipo de prueba
- Directriz Derivar Caso de Prueba de CU
- CREG Course Registration System (RUP)
 - Supplementary Specification v1 - Course Registration System.pdf
 - Test Plan Course Registration System.pdf

Preguntas?

